Experiment 'p_14_7_bubble_sort_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_14_7_bubble_sort_array
Natural language statement: Write a method that implements the bubble sort algorithm to sort an array of integers.
Method signature: p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)

Ensures

- `forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]`
- `multiset(inputs[..]) == multiset(sorted[..])`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that implements the bubble sort algorithm to sort an array of integers.

The signature should be:

p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures
    ↪ multiset(inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnKrTAOf1UgpTG9VstBb1NExZOz6v
Model: gpt-5.5-2026-04-23
Created at: 1780650635
Finish reason: stop
Usage: 324 tokens in, and 9311 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n];

  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant sorted.Length == n
    invariant forall t :: 0 <= t < k ==> sorted[t] == inputs[t]
    decreases n - k
  {
    sorted[k] := inputs[k];
    k := k + 1;
  }

  assert forall t | 0 <= t < n :: sorted[t] == inputs[t];
  assert |sorted[..]| == |inputs[..]|;
  assert forall t :: 0 <= t < |sorted[..]| ==> sorted[..][t] == inputs[..][t];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);
```

```
var end := n;
while 1 < end
  invariant 0 <= end <= n
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall i, j :: end <= i < j < n ==> sorted[i] <= sorted[j]
  invariant forall i, j :: 0 <= i < end <= j < n ==> sorted[i] <= sorted[j]
  decreases end
{
  var j := 0;
  while j + 1 < end
    invariant 0 <= j < end <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall i, k :: end <= i < k < n ==> sorted[i] <= sorted[k]
    invariant forall i, k :: 0 <= i < end <= k < n ==> sorted[i] <= sorted[k]
    invariant forall i :: 0 <= i <= j ==> sorted[i] <= sorted[j]
    decreases end - j
  {
    if sorted[j] > sorted[j + 1] {
      ghost var s0 := sorted[..];
      assert |s0| == n;
      assert s0[j] > s0[j + 1];
      assert multiset(s0) == multiset(inputs[..]);
      assert forall p | 0 <= p <= j :: s0[p] <= s0[j];
      assert forall p, q | end <= p < q < n :: s0[p] <= s0[q];
      assert forall p, q | 0 <= p < end <= q < n :: s0[p] <= s0[q];

      var tmp := sorted[j];
      sorted[j] := sorted[j + 1];
      sorted[j + 1] := tmp;

      assert j + 2 <= n;
      assert s0 == s0[..j] + [s0[j]] + [s0[j + 1]] + s0[j + 2..];
      assert sorted[..j] == s0[..j];
      assert sorted[j] == s0[j + 1];
      assert sorted[j + 1] == s0[j];
      assert sorted[j + 2..] == s0[j + 2..];
      assert sorted[..] == s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..];
      calc {
        multiset(sorted[..]);
      == {
        assert sorted[..] == s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..];
      }
        multiset(s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..]);
      == { }
        multiset(s0[..j]) + multiset([s0[j + 1]]) + multiset([s0[j]]) + multiset(s0[j +
            ↪ 2..]);
      == { }
        multiset(s0[..j]) + multiset([s0[j]]) + multiset([s0[j + 1]]) + multiset(s0[j +
```

```
      ↪ 2..]);
  == {
    assert s0 == s0[..j] + [s0[j]] + [s0[j + 1]] + s0[j + 2..];
  }
    multiset(s0);
  == {
    assert multiset(s0) == multiset(inputs[..]);
  }
    multiset(inputs[..]);
  }

  assert forall p | 0 <= p <= j + 1 :: sorted[p] <= sorted[j + 1] by {
    if p < j {
      assert sorted[p] == s0[p];
      assert s0[p] <= s0[j];
      assert sorted[j + 1] == s0[j];
    } else if p == j {
      assert sorted[p] == s0[j + 1];
      assert sorted[j + 1] == s0[j];
      assert s0[j + 1] < s0[j];
    } else {
      assert p == j + 1;
    }
  }
  assert forall p, q | end <= p < q < n :: sorted[p] <= sorted[q] by {
    assert p != j && p != j + 1;
    assert q != j && q != j + 1;
    assert sorted[p] == s0[p];
    assert sorted[q] == s0[q];
    assert s0[p] <= s0[q];
  }
  assert forall p, q | 0 <= p < end <= q < n :: sorted[p] <= sorted[q] by {
    assert q != j && q != j + 1;
    assert sorted[q] == s0[q];
    if p == j {
      assert sorted[p] == s0[j + 1];
      assert s0[j + 1] <= s0[q];
    } else if p == j + 1 {
      assert sorted[p] == s0[j];
      assert s0[j] <= s0[q];
    } else {
      assert sorted[p] == s0[p];
      assert s0[p] <= s0[q];
    }
  }
  assert multiset(sorted[..]) == multiset(inputs[..]);
} else {
  assert sorted[j] <= sorted[j + 1];
  assert forall p | 0 <= p <= j + 1 :: sorted[p] <= sorted[j + 1] by {
    if p <= j {
      assert sorted[p] <= sorted[j];
```

```
        assert sorted[j] <= sorted[j + 1];
      } else {
        assert p == j + 1;
      }
    }
    assert forall p, q | end <= p < q < n :: sorted[p] <= sorted[q];
    assert forall p, q | 0 <= p < end <= q < n :: sorted[p] <= sorted[q];
    assert multiset(sorted[..]) == multiset(inputs[..]);
  }

  j := j + 1;
 }

 assert j == end - 1;
 assert forall p, q | end - 1 <= p < q < n :: sorted[p] <= sorted[q] by {
  if p == end - 1 {
    assert p == j;
    assert end <= q;
    assert 0 <= p < end <= q < n;
    assert sorted[p] <= sorted[q];
  } else {
    assert end <= p;
    assert sorted[p] <= sorted[q];
  }
 }
 assert forall p, q | 0 <= p < end - 1 <= q < n :: sorted[p] <= sorted[q] by {
  if q == end - 1 {
    assert q == j;
    assert 0 <= p <= j;
    assert sorted[p] <= sorted[j];
  } else {
    assert end <= q;
    assert p < end;
    assert sorted[p] <= sorted[q];
  }
 }

 end := end - 1;
}

assert forall i, j | 0 <= i < j < n :: sorted[i] <= sorted[j] by {
 if end <= i {
  assert sorted[i] <= sorted[j];
 } else {
  assert i < end;
  assert end <= 1;
  if end == 0 {
    assert !(0 <= i < end);
  } else {
    assert end == 1;
    assert i == 0;
```

```
      assert end <= j;
      assert sorted[i] <= sorted[j];
    }
   }
  }
  assert sorted.Length == n;
  assert multiset(sorted[..]) == multiset(inputs[..]);
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_7_bubble_sort_array/program.dfy(85,13): Error: unresolved identifier: p
   |
85 | if p < j {
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(86,26): Error: unresolved identifier: p
   |
86 | assert sorted[p] == s0[p];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(86,35): Error: unresolved identifier: p
   |
86 | assert sorted[p] == s0[p];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(87,22): Error: unresolved identifier: p
   |
87 | assert s0[p] <= s0[j];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(89,20): Error: unresolved identifier: p
   |
89 | } else if p == j {
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(90,26): Error: unresolved identifier: p
   |
90 | assert sorted[p] == s0[j + 1];
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(94,19): Error: unresolved identifier: p
   |
94 | assert p == j + 1;
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(98,17): Error: unresolved identifier: p
   |
98 | assert p != j && p != j + 1;
```

```
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(98,27): Error: unresolved identifier: p
   |
98 | assert p != j && p != j + 1;
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(99,17): Error: unresolved identifier: q
   |
99 | assert q != j && q != j + 1;
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(99,27): Error: unresolved identifier: q
   |
99 | assert q != j && q != j + 1;
   | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(100,24): Error: unresolved identifier:
    ↪ p
    |
100 | assert sorted[p] == s0[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(100,33): Error: unresolved identifier:
    ↪ p
    |
100 | assert sorted[p] == s0[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(101,24): Error: unresolved identifier:
    ↪ q
    |
101 | assert sorted[q] == s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(101,33): Error: unresolved identifier:
    ↪ q
    |
101 | assert sorted[q] == s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(102,20): Error: unresolved identifier:
    ↪ p
    |
102 | assert s0[p] <= s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(102,29): Error: unresolved identifier:
    ↪ q
    |
102 | assert s0[p] <= s0[q];
```

```
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(105,17): Error: unresolved identifier:
    ↪ q
    |
105 | assert q != j && q != j + 1;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(105,27): Error: unresolved identifier:
    ↪ q
    |
105 | assert q != j && q != j + 1;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(106,24): Error: unresolved identifier:
    ↪ q
    |
106 | assert sorted[q] == s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(106,33): Error: unresolved identifier:
    ↪ q
    |
106 | assert sorted[q] == s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(107,13): Error: unresolved identifier:
    ↪ p
    |
107 | if p == j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(108,26): Error: unresolved identifier:
    ↪ p
    |
108 | assert sorted[p] == s0[j + 1];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(109,35): Error: unresolved identifier:
    ↪ q
    |
109 | assert s0[j + 1] <= s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(110,20): Error: unresolved identifier:
    ↪ p
    |
110 | } else if p == j + 1 {
    | ^
```

```
problem_data/p_14_7_bubble_sort_array/program.dfy(111,26): Error: unresolved identifier:
    ↪ p
    |
111 | assert sorted[p] == s0[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(112,31): Error: unresolved identifier:
    ↪ q
    |
112 | assert s0[j] <= s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(114,26): Error: unresolved identifier:
    ↪ p
    |
114 | assert sorted[p] == s0[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(114,35): Error: unresolved identifier:
    ↪ p
    |
114 | assert sorted[p] == s0[p];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(115,22): Error: unresolved identifier:
    ↪ p
    |
115 | assert s0[p] <= s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(115,31): Error: unresolved identifier:
    ↪ q
    |
115 | assert s0[p] <= s0[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(122,13): Error: unresolved identifier:
    ↪ p
    |
122 | if p <= j {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(123,26): Error: unresolved identifier:
    ↪ p
    |
123 | assert sorted[p] <= sorted[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(126,19): Error: unresolved identifier:
    ↪ p
    |
```

```
126 | assert p == j + 1;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(139,9): Error: unresolved identifier: p
    |
139 | if p == end - 1 {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(140,15): Error: unresolved identifier:
    ↪ p
    |
140 | assert p == j;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(141,22): Error: unresolved identifier:
    ↪ q
    |
141 | assert end <= q;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(142,20): Error: unresolved identifier:
    ↪ p
    |
142 | assert 0 <= p < end <= q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(142,31): Error: unresolved identifier:
    ↪ q
    |
142 | assert 0 <= p < end <= q < n;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(143,22): Error: unresolved identifier:
    ↪ p
    |
143 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(143,35): Error: unresolved identifier:
    ↪ q
    |
143 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(145,22): Error: unresolved identifier:
    ↪ p
    |
145 | assert end <= p;
    | ^
```

```
problem_data/p_14_7_bubble_sort_array/program.dfy(146,22): Error: unresolved identifier:
    ↪ p
    |
146 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(146,35): Error: unresolved identifier:
    ↪ q
    |
146 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(150,9): Error: unresolved identifier: q
    |
150 | if q == end - 1 {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(151,15): Error: unresolved identifier:
    ↪ q
    |
151 | assert q == j;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(152,20): Error: unresolved identifier:
    ↪ p
    |
152 | assert 0 <= p <= j;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(153,22): Error: unresolved identifier:
    ↪ p
    |
153 | assert sorted[p] <= sorted[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(155,22): Error: unresolved identifier:
    ↪ q
    |
155 | assert end <= q;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(156,15): Error: unresolved identifier:
    ↪ p
    |
156 | assert p < end;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(157,22): Error: unresolved identifier:
    ↪ p
    |
157 | assert sorted[p] <= sorted[q];
```

```
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(157,35): Error: unresolved identifier:
    ↪ q
    |
157 | assert sorted[p] <= sorted[q];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(165,14): Error: unresolved identifier:
    ↪ i
    |
165 | if end <= i {
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(166,20): Error: unresolved identifier:
    ↪ i
    |
166 | assert sorted[i] <= sorted[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(166,33): Error: unresolved identifier:
    ↪ j
    |
166 | assert sorted[i] <= sorted[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(168,13): Error: unresolved identifier:
    ↪ i
    |
168 | assert i < end;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(171,22): Error: unresolved identifier:
    ↪ i
    |
171 | assert !(0 <= i < end);
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(174,15): Error: unresolved identifier:
    ↪ i
    |
174 | assert i == 0;
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(175,22): Error: unresolved identifier:
    ↪ j
    |
175 | assert end <= j;
    | ^
```

```
problem_data/p_14_7_bubble_sort_array/program.dfy(176,22): Error: unresolved identifier:
    ↪ i
    |
176 | assert sorted[i] <= sorted[j];
    | ^

problem_data/p_14_7_bubble_sort_array/program.dfy(176,35): Error: unresolved identifier:
    ↪ j
    |
176 | assert sorted[i] <= sorted[j];
    | ^

61 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnKughdmmB9z7qCaFbYeZtqMpmKVB
Model: gpt-5.5-2026-04-23
Created at: 1780650834
Finish reason: stop
Usage: 7792 tokens in, and 3262 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n];

  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant sorted.Length == n
    invariant forall t :: 0 <= t < k ==> sorted[t] == inputs[t]
    decreases n - k
  {
    sorted[k] := inputs[k];
    k := k + 1;
  }

  assert forall t | 0 <= t < n :: sorted[t] == inputs[t];
  assert |sorted[..]| == |inputs[..]|;
  assert forall t :: 0 <= t < |sorted[..]| ==> sorted[..][t] == inputs[..][t];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var end := n;
  while 1 < end
    invariant 0 <= end <= n
```

```
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall i, j :: end <= i < j < n ==> sorted[i] <= sorted[j]
  invariant forall i, j :: 0 <= i < end <= j < n ==> sorted[i] <= sorted[j]
  decreases end
{
  var j := 0;
  while j + 1 < end
    invariant 0 <= j < end <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall i, k :: end <= i < k < n ==> sorted[i] <= sorted[k]
    invariant forall i, k :: 0 <= i < end <= k < n ==> sorted[i] <= sorted[k]
    invariant forall i :: 0 <= i <= j ==> sorted[i] <= sorted[j]
    decreases end - j
  {
    if sorted[j] > sorted[j + 1] {
      ghost var s0 := sorted[..];
      assert |s0| == n;
      assert s0[j] > s0[j + 1];
      assert multiset(s0) == multiset(inputs[..]);
      assert forall p | 0 <= p <= j :: s0[p] <= s0[j];
      assert forall p, q | end <= p < q < n :: s0[p] <= s0[q];
      assert forall p, q | 0 <= p < end <= q < n :: s0[p] <= s0[q];

      var tmp := sorted[j];
      sorted[j] := sorted[j + 1];
      sorted[j + 1] := tmp;

      assert j + 2 <= n;
      assert s0 == s0[..j] + [s0[j]] + [s0[j + 1]] + s0[j + 2..];
      assert sorted[..j] == s0[..j];
      assert sorted[j] == s0[j + 1];
      assert sorted[j + 1] == s0[j];
      assert sorted[j + 2..] == s0[j + 2..];
      assert sorted[..] == s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..];
      calc {
        multiset(sorted[..]);
      == {
        assert sorted[..] == s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..];
      }
        multiset(s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..]);
      == { }
        multiset(s0[..j]) + multiset([s0[j + 1]]) + multiset([s0[j]]) + multiset(s0[j +
            ↪ 2..]);
      == { }
        multiset(s0[..j]) + multiset([s0[j]]) + multiset([s0[j + 1]]) + multiset(s0[j +
            ↪ 2..]);
      == {
        assert s0 == s0[..j] + [s0[j]] + [s0[j + 1]] + s0[j + 2..];
      }
```

```
      multiset(s0);
    == {
      assert multiset(s0) == multiset(inputs[..]);
    }
      multiset(inputs[..]);
    }

    forall p | 0 <= p <= j + 1
      ensures sorted[p] <= sorted[j + 1]
    {
      if p < j {
        assert sorted[p] == s0[p];
        assert s0[p] <= s0[j];
        assert sorted[j + 1] == s0[j];
      } else if p == j {
        assert sorted[p] == s0[j + 1];
        assert sorted[j + 1] == s0[j];
        assert s0[j + 1] < s0[j];
      } else {
        assert p == j + 1;
      }
    }
    forall p, q | end <= p < q < n
      ensures sorted[p] <= sorted[q]
    {
      assert p != j && p != j + 1;
      assert q != j && q != j + 1;
      assert sorted[p] == s0[p];
      assert sorted[q] == s0[q];
      assert s0[p] <= s0[q];
    }
    forall p, q | 0 <= p < end <= q < n
      ensures sorted[p] <= sorted[q]
    {
      assert q != j && q != j + 1;
      assert sorted[q] == s0[q];
      if p == j {
        assert sorted[p] == s0[j + 1];
        assert s0[j + 1] <= s0[q];
      } else if p == j + 1 {
        assert sorted[p] == s0[j];
        assert s0[j] <= s0[q];
      } else {
        assert sorted[p] == s0[p];
        assert s0[p] <= s0[q];
      }
    }
    assert multiset(sorted[..]) == multiset(inputs[..]);
  } else {
    assert sorted[j] <= sorted[j + 1];
    forall p | 0 <= p <= j + 1
```

```
      ensures sorted[p] <= sorted[j + 1]
    {
      if p <= j {
        assert sorted[p] <= sorted[j];
        assert sorted[j] <= sorted[j + 1];
      } else {
        assert p == j + 1;
      }
    }
    assert forall p, q | end <= p < q < n :: sorted[p] <= sorted[q];
    assert forall p, q | 0 <= p < end <= q < n :: sorted[p] <= sorted[q];
    assert multiset(sorted[..]) == multiset(inputs[..]);
  }

  j := j + 1;
}

assert j == end - 1;
forall p, q | end - 1 <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  if p == end - 1 {
    assert p == j;
    assert end <= q;
    assert 0 <= p < end <= q < n;
    assert sorted[p] <= sorted[q];
  } else {
    assert end <= p;
    assert sorted[p] <= sorted[q];
  }
}
forall p, q | 0 <= p < end - 1 <= q < n
  ensures sorted[p] <= sorted[q]
{
  if q == end - 1 {
    assert q == j;
    assert 0 <= p <= j;
    assert sorted[p] <= sorted[j];
  } else {
    assert end <= q;
    assert p < end;
    assert sorted[p] <= sorted[q];
  }
}

end := end - 1;
}

forall i, j | 0 <= i < j < n
  ensures sorted[i] <= sorted[j]
{
```

```
    if end <= i {
      assert sorted[i] <= sorted[j];
    } else {
      assert i < end;
      assert end <= 1;
      if end == 0 {
        assert !(0 <= i < end);
      } else {
        assert end == 1;
        assert i == 0;
        assert end <= j;
        assert sorted[i] <= sorted[j];
      }
    }
  }
  assert sorted.Length == n;
  assert multiset(sorted[..]) == multiset(inputs[..]);
}
```

Final Program

```
method p_14_4_bubble_sort_array(inputs: array<int>) returns (sorted: array<int>)
       ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
       ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var n := inputs.Length;
  sorted := new int[n];

  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant sorted.Length == n
    invariant forall t :: 0 <= t < k ==> sorted[t] == inputs[t]
    decreases n - k
  {
    sorted[k] := inputs[k];
    k := k + 1;
  }

  assert forall t | 0 <= t < n :: sorted[t] == inputs[t];
  assert |sorted[..]| == |inputs[..]|;
  assert forall t :: 0 <= t < |sorted[..]| ==> sorted[..][t] == inputs[..][t];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var end := n;
  while 1 < end
    invariant 0 <= end <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
```

```
  invariant forall i, j :: end <= i < j < n ==> sorted[i] <= sorted[j]
  invariant forall i, j :: 0 <= i < end <= j < n ==> sorted[i] <= sorted[j]
  decreases end
{
  var j := 0;
  while j + 1 < end
    invariant 0 <= j < end <= n
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall i, k :: end <= i < k < n ==> sorted[i] <= sorted[k]
    invariant forall i, k :: 0 <= i < end <= k < n ==> sorted[i] <= sorted[k]
    invariant forall i :: 0 <= i <= j ==> sorted[i] <= sorted[j]
    decreases end - j
  {
    if sorted[j] > sorted[j + 1] {
      ghost var s0 := sorted[..];
      assert |s0| == n;
      assert s0[j] > s0[j + 1];
      assert multiset(s0) == multiset(inputs[..]);
      assert forall p | 0 <= p <= j :: s0[p] <= s0[j];
      assert forall p, q | end <= p < q < n :: s0[p] <= s0[q];
      assert forall p, q | 0 <= p < end <= q < n :: s0[p] <= s0[q];

      var tmp := sorted[j];
      sorted[j] := sorted[j + 1];
      sorted[j + 1] := tmp;

      assert j + 2 <= n;
      assert s0 == s0[..j] + [s0[j]] + [s0[j + 1]] + s0[j + 2..];
      assert sorted[..j] == s0[..j];
      assert sorted[j] == s0[j + 1];
      assert sorted[j + 1] == s0[j];
      assert sorted[j + 2..] == s0[j + 2..];
      assert sorted[..] == s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..];
      calc {
        multiset(sorted[..]);
      == {
        assert sorted[..] == s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..];
      }
        multiset(s0[..j] + [s0[j + 1]] + [s0[j]] + s0[j + 2..]);
      == { }
        multiset(s0[..j]) + multiset([s0[j + 1]]) + multiset([s0[j]]) + multiset(s0[j +
            ↪ 2..]);
      == { }
        multiset(s0[..j]) + multiset([s0[j]]) + multiset([s0[j + 1]]) + multiset(s0[j +
            ↪ 2..]);
      == {
        assert s0 == s0[..j] + [s0[j]] + [s0[j + 1]] + s0[j + 2..];
      }
        multiset(s0);
      == {
```

```
      assert multiset(s0) == multiset(inputs[..]);
    }
      multiset(inputs[..]);
    }

    forall p | 0 <= p <= j + 1
      ensures sorted[p] <= sorted[j + 1]
    {
      if p < j {
        assert sorted[p] == s0[p];
        assert s0[p] <= s0[j];
        assert sorted[j + 1] == s0[j];
      } else if p == j {
        assert sorted[p] == s0[j + 1];
        assert sorted[j + 1] == s0[j];
        assert s0[j + 1] < s0[j];
      } else {
        assert p == j + 1;
      }
    }
    forall p, q | end <= p < q < n
      ensures sorted[p] <= sorted[q]
    {
      assert p != j && p != j + 1;
      assert q != j && q != j + 1;
      assert sorted[p] == s0[p];
      assert sorted[q] == s0[q];
      assert s0[p] <= s0[q];
    }
    forall p, q | 0 <= p < end <= q < n
      ensures sorted[p] <= sorted[q]
    {
      assert q != j && q != j + 1;
      assert sorted[q] == s0[q];
      if p == j {
        assert sorted[p] == s0[j + 1];
        assert s0[j + 1] <= s0[q];
      } else if p == j + 1 {
        assert sorted[p] == s0[j];
        assert s0[j] <= s0[q];
      } else {
        assert sorted[p] == s0[p];
        assert s0[p] <= s0[q];
      }
    }
    assert multiset(sorted[..]) == multiset(inputs[..]);
  } else {
    assert sorted[j] <= sorted[j + 1];
    forall p | 0 <= p <= j + 1
      ensures sorted[p] <= sorted[j + 1]
    {
```

```
      if p <= j {
        assert sorted[p] <= sorted[j];
        assert sorted[j] <= sorted[j + 1];
      } else {
        assert p == j + 1;
      }
    }
    assert forall p, q | end <= p < q < n :: sorted[p] <= sorted[q];
    assert forall p, q | 0 <= p < end <= q < n :: sorted[p] <= sorted[q];
    assert multiset(sorted[..]) == multiset(inputs[..]);
  }

  j := j + 1;
}

assert j == end - 1;
forall p, q | end - 1 <= p < q < n
  ensures sorted[p] <= sorted[q]
{
  if p == end - 1 {
    assert p == j;
    assert end <= q;
    assert 0 <= p < end <= q < n;
    assert sorted[p] <= sorted[q];
  } else {
    assert end <= p;
    assert sorted[p] <= sorted[q];
  }
}
forall p, q | 0 <= p < end - 1 <= q < n
  ensures sorted[p] <= sorted[q]
{
  if q == end - 1 {
    assert q == j;
    assert 0 <= p <= j;
    assert sorted[p] <= sorted[j];
  } else {
    assert end <= q;
    assert p < end;
    assert sorted[p] <= sorted[q];
  }
}

end := end - 1;
}

forall i, j | 0 <= i < j < n
  ensures sorted[i] <= sorted[j]
{
  if end <= i {
    assert sorted[i] <= sorted[j];
```

```
    } else {
      assert i < end;
      assert end <= 1;
      if end == 0 {
        assert !(0 <= i < end);
      } else {
        assert end == 1;
        assert i == 0;
        assert end <= j;
        assert sorted[i] <= sorted[j];
      }
    }
  }
  assert sorted.Length == n;
  assert multiset(sorted[..]) == multiset(inputs[..]);
}
```

Total Token Usage

Input tokens: 8116
Output tokens: 12573
Reasoning tokens: 8416
Sum of 'total tokens': 20689

Experiment Timings

Overall Experiment started at 1780650634731, ended at 1780650934643, lasting 299912ms (299.91 seconds)
Iteration #1 started at 1780650634732, ended at 1780650832196, lasting 197464ms (197.46 seconds)
Iteration #2 started at 1780650832196, ended at 1780650934643, lasting 102447ms (102.45 seconds)